

Aperture — Low-Level Design

Module: Discovery

Version: v1.0

Module Owner: Discovery Platform

Dependencies:

- Users Module
 - Metadata Module
 - Reviews Module
 - Lists Module
 - Search Module
 - Streaming Module
 - AI Module
 - Analytics Module (future)
-

1. Overview

The Discovery module is responsible for helping users find content they are likely to enjoy.

Unlike the Metadata module, which stores content, or the Search module, which retrieves content on demand, the Discovery module proactively surfaces relevant content by combining multiple independent signals.

Its primary responsibility is orchestration rather than ownership.

Discovery does not create or own movies, reviews, ratings, or lists. Instead, it composes those domains into meaningful user experiences.

2. Design Goals

The Discovery module should:

- Maximize meaningful content discovery
- Personalize without creating filter bubbles
- Blend editorial and algorithmic recommendations
- Support experimentation with ranking strategies

- Remain explainable and debuggable
 - Gracefully evolve from rule-based to AI-assisted recommendations
-

3. Module Ownership

Owns

- Discovery feeds
 - Recommendation orchestration
 - Trending calculations
 - Editorial collections
 - Homepage composition
 - Ranking strategies
 - Feed configuration
-

Reads

- Metadata
 - Reviews
 - Ratings
 - Lists
 - User preferences
 - Streaming availability
 - AI recommendation outputs
-

Never Modifies

- Movies
- TV Shows
- Reviews
- Lists
- User profiles
- Ratings

Discovery consumes data but does not own or mutate it.

4. Responsibilities

The Discovery module is responsible for generating:

Homepage

Examples:

- Trending Now
 - New Releases
 - Continue Exploring
 - Because You Liked...
 - Editor's Picks
-

Recommendation Collections

Examples:

- Similar Titles
 - Hidden Gems
 - Award Winners
 - Popular This Week
 - Underrated Classics
-

Editorial Collections

Curated by administrators.

Examples:

- Best Neo-Noir Films
 - Women Directors Spotlight
 - Summer Blockbusters
 - Cannes Winners
-

Personalized Discovery

Future examples:

- Weekend Recommendations
 - Mood-Based Discovery
 - Director Journeys
 - Genre Evolution
-

5. High-Level Workflow

Homepage Request



Load User Context



Determine Feed Configuration



Collect Candidate Content



Rank Candidates



Apply Diversity Rules



Remove Duplicates



Assemble Homepage



Return Response

6. Data Flow

User



Discovery API



Discovery Service



Metadata



Reviews



Lists



Streaming



AI Signals



Ranking Engine



Homepage Response

The Discovery module acts as an orchestrator that combines outputs from multiple independent modules.

7. Feed Composition

Every feed is composed from multiple candidate pools.

Candidate sources may include:

- Trending

- Popular
- Recently Released
- Similar Titles
- Editorial Picks
- Community Favorites
- Personalized Suggestions

The feed generator merges these pools according to configurable ranking rules.

8. Recommendation Strategy

Recommendation quality evolves through stages.

Stage 1

Rule-based recommendations

Signals:

- Genres
 - Directors
 - Cast
 - Keywords
-

Stage 2

Behavior-based recommendations

Signals:

- Ratings
 - Reviews
 - Watchlists
 - Favorites
 - Lists
-

Stage 3

Hybrid recommendations

Combine:

- Metadata similarity
- Behavioral similarity
- Popularity
- Editorial signals
- Embedding similarity

The recommendation algorithm should remain replaceable without affecting clients.

9. Trending Algorithm

Trending is intentionally different from "most popular."

Potential signals include:

- Review velocity
- Rating velocity
- List additions
- Search frequency
- Community engagement
- Recent releases

This allows trending to reflect current interest rather than historical popularity.

10. Diversity Strategy

Repeated recommendations reduce discovery quality.

The Discovery module should intentionally diversify recommendations by balancing:

- Genres
- Release years
- Countries
- Languages
- Directors
- Popularity

The goal is to increase exploration while maintaining relevance.

11. Personalization

User context may include:

- Favorite genres
- Favorite directors
- Highly rated titles
- Viewing history
- Preferred languages
- Preferred runtime
- Streaming subscriptions (future)

Personalization should remain transparent and adjustable.

12. Editorial Integration

Editors should be able to influence discovery without modifying algorithms.

Examples:

- Featured collections
- Seasonal recommendations
- Festival highlights
- Staff picks

Editorial content complements algorithmic recommendations rather than replacing them.

13. AI Integration

The Discovery module consumes AI outputs but does not generate them.

Examples:

- Cinematic DNA
- Emotional similarity
- Taste matching
- Semantic recommendations

This separation allows AI systems to evolve independently.

14. Caching Strategy

Cache candidates:

- Homepage sections
- Trending feeds
- Editorial collections
- Popular recommendations

Highly personalized feeds should use shorter cache lifetimes than global feeds.

15. Performance Considerations

Potential bottlenecks:

- Homepage assembly
- Multiple downstream requests
- Personalized ranking
- Large recommendation pools

Mitigation strategies:

- Parallel data retrieval
- Cached feed sections
- Background recommendation generation
- Precomputed rankings

The Discovery module should prioritize perceived responsiveness over perfect freshness.

16. Security Considerations

Discovery should respect:

- Content visibility
- User privacy settings
- Regional availability
- Age restrictions (future)

Private user activity should never influence public recommendations in identifiable ways.

17. Error Handling

Representative failures:

- Recommendation backend unavailable
- Missing personalization data
- Metadata unavailable
- AI service timeout

Fallback behavior:

- Return editorial collections
- Return trending content
- Return popular titles

The homepage should remain useful even when personalization fails.

18. Testing Strategy

Unit Tests

- Ranking logic
 - Diversity rules
 - Feed composition
 - Candidate selection
-

Integration Tests

- Metadata integration
 - Review signal integration
 - Streaming integration
 - AI integration
-

API Tests

- Homepage endpoints
 - Recommendation endpoints
 - Trending endpoints
-

Performance Tests

Measure:

- Homepage latency
 - Recommendation generation
 - Feed assembly
 - Cache effectiveness
-

19. Future Service Extraction

The Discovery module is one of the strongest candidates for future extraction.

Reasons:

- High read volume
- Independent scaling characteristics
- Complex orchestration
- AI integration
- Recommendation workloads

Potential published events:

- DiscoveryFeedGenerated
- RecommendationServed

Potential consumed events:

- ReviewCreated
- RatingChanged
- ListUpdated
- MetadataUpdated
- UserPreferenceChanged

The module is intentionally designed to evolve into an independently deployable service if future scale requires it.

20. Future Evolution

Potential enhancements:

- Emotional Discovery Engine
- Natural language discovery
- Dynamic seasonal collections

- AI-generated editorial pages
- Friend-based recommendations
- Collaborative discovery
- Explainable recommendations
- Cross-media discovery

The architecture should support these additions without changing client-facing APIs.

21. Interview Discussion Points

Be prepared to explain:

- Why separate Discovery from Search?
 - Why should Discovery own orchestration instead of recommendation algorithms?
 - How do you avoid filter bubbles?
 - How would you balance editorial and algorithmic recommendations?
 - Why should Discovery not own any persistent business data?
 - How would you scale homepage generation for millions of users?
-

22. Summary

The Discovery module is the orchestration layer that transforms Aperture from a content database into an intelligent discovery platform.

By combining metadata, community activity, personalization, editorial curation, and AI-generated insights, it creates a flexible, scalable, and explainable discovery experience while maintaining clean boundaries between business domains.